

Diseño Proyecto 1 - Desarrollo de una aplicación para la generación de gráficos y texto

Kevin Lobo Juárez

2020087823

Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica

I. REQUERIMIENTOS DEL SISTEMA

Este proyecto consiste en el desarrollo de una aplicación que permite procesar imágenes en escala de grises mediante interpolación bilineal, utilizando una combinación de Python y lenguaje ensamblador. A continuación, se describen los requerimientos detallados que guían su desarrollo.

A. Requerimientos funcionales

- El sistema debe ser capaz de convertir imágenes en formato PNG a un formato binario (.img) en escala de grises (valores entre 0 y 255).
- La imagen original debe tener un tamaño mínimo de 390x390 píxeles para garantizar que se pueda extraer un cuadrante de 180x170 sin pérdida de información.
- Se debe seleccionar un cuadrante de 180x170 píxeles de la imagen para su posterior procesamiento.
- El cuadrante debe ser interpolado mediante interpolación bilineal hasta alcanzar un tamaño de 360x340 píxeles.
- El procesamiento de interpolación debe realizarse en lenguaje ensamblador (NASM x86-64, ARM o RISC-V, según la propuesta).
- El resultado procesado debe ser convertido nuevamente a formato PNG para su visualización.
- Todo el sistema debe ejecutarse de manera compatible con Linux (versión 24).

B. Estado del Arte y Estándares

1) *Interpolación Bilineal*: La interpolación bilineal es una técnica estándar en procesamiento de imágenes utilizada para escalar imágenes y suavizar transiciones entre píxeles. Consiste en calcular el valor de un píxel desconocido como una ponderación lineal de los cuatro píxeles más cercanos en una cuadrícula. Este método es ampliamente utilizado por su balance entre simplicidad computacional y calidad visual [1].

2) *Estándares de Imágenes*: Para asegurar la portabilidad y compatibilidad, el proyecto utiliza el formato de imagen .png (Portable Network Graphics), un estándar reconocido por la ISO (ISO/IEC 15948:2004). Este formato ofrece compresión sin pérdida y soporte para transparencia, lo que lo hace ideal para almacenamiento y visualización de imágenes procesadas [2].

3) *Lenguaje Ensamblador (NASM)*: El ensamblador NASM (Netwide Assembler) es una herramienta ampliamente utilizada para el desarrollo de software de bajo nivel en arquitecturas x86 y x86-64. NASM permite un control directo sobre el hardware, optimizando tareas críticas como la interpolación bilineal [3]. El uso de NASM en este proyecto asegura un alto rendimiento en la interpolación bilineal, aprovechando al máximo los recursos de hardware disponibles debido a que se está usando x86 de arquitectura nativa.

II. PROPUESTAS DE SOLUCIÓN AL PROBLEMA

Descripción General del Problema: El problema planteado consiste en implementar un sistema que permita procesar imágenes mediante interpolación bilineal en un cuadrante seleccionado por el usuario. El resultado debe ser almacenado en un archivo y visualizado posteriormente en formato PNG. Para esta tarea se han considerado tres opciones de solución que combinan diferentes arquitecturas y herramientas de programación: **x86 NASM y Python, RISC-V rv8 y Python, y ARM arm-none-eabi-gcc y Python.**

1) *Opción 1: x86 NASM y Python*: Esta opción utiliza ensamblador NASM para implementar la interpolación bilineal directamente en hardware x86, aprovechando la arquitectura nativa de la laptop. Python se utiliza para la selección de cuadrantes y la conversión de archivos de salida a PNG como se puede visualizar en la figura 1.

Ventajas:

- **Compatibilidad nativa:** No requiere emuladores, ya que la arquitectura x86-64 es nativa de la máquina.
- **Rendimiento:** El código ensamblador aprovecha la velocidad del hardware directamente.
- **Facilidad en manejo de archivos:** NASM permite realizar operaciones de archivos de manera sencilla con llamadas al sistema.

Desventajas:

- **Complejidad del ensamblador:** NASM tiene muchas instrucciones y requiere un conocimiento profundo para implementar operaciones complejas.

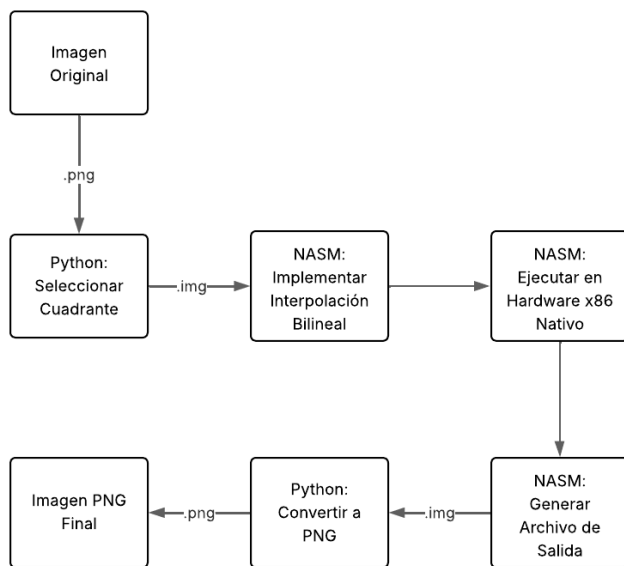


Fig. 1. Diagrama de flujo de la implementación de NASM + PYTHON.

El ensamblador realiza las operaciones críticas directamente en la arquitectura nativa, mientras que Python maneja las tareas de alto nivel.

2) *Opción 2: RISC-V rv8 y Python:* En esta opción, se utiliza el ensamblador para la arquitectura RISC-V mediante la emulación con rv8. Python se emplea para las tareas de selección de cuadrantes y conversión a PNG. Esta opción requiere un flujo adicional para filtrar manualmente los datos de salida generados por RISC-V como se puede visualizar en la figura 2.

Ventajas:

- **Arquitectura moderna:** RISC-V es un estándar abierto y de rápida adopción en el ámbito académico e industrial.
- **Modularidad:** La segmentación en Python y RISC-V permite dividir las responsabilidades del sistema.

Desventajas:

- **Uso de emuladores:** La emulación introduce latencias y problemas de rendimiento, especialmente cuando se realizan cambios en los registros.
- **Filtración de datos:** Los datos generados no están estructurados, lo que requiere una filtración manual adicional para ser procesados correctamente.
- **Complejidad técnica:** El uso de RISC-V y la emulación requieren configuraciones específicas y un manejo avanzado del toolchain.

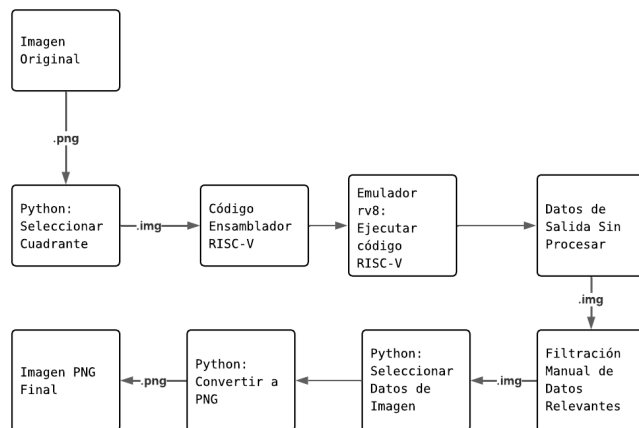


Fig. 2. Diagrama de flujo de la implementación de RISC-V + PYTHON.

Utiliza un emulador RISC-V (rv8) para ejecutar ensamblador, pero requiere filtración manual de datos se requiere de un operador de redirección de salida ";" para indicar en el archivo que se quiere guardar los datos.

3) *Opción 3: ARM arm-none-eabi-gcc y Python:* Esta opción emplea la arquitectura ARM, utilizando el compilador arm-none-eabi-gcc para desarrollar el ensamblador. Python se utiliza como soporte para la selección de cuadrantes y la conversión de resultados a PNG. Esta opción requiere configurar un entorno cruzado para ensamblar y ejecutar el código en ARM como se puede visualizar en la figura 3.

Ventajas:

- **Simplicidad en instrucciones:** ARM tiene un conjunto reducido de instrucciones, lo que simplifica el desarrollo.
- **Popularidad:** ARM es ampliamente utilizado en sistemas embebidos, lo que facilita encontrar recursos y documentación.

Desventajas:

- **Configuración compleja:** Requiere configurar un entorno cruzado con arm-none-eabi-gcc y qemu, lo que añade complejidad al proceso.
- **Manejo de archivos:** El ensamblador ARM no proporciona un manejo de archivos directo, lo que requiere procesar los datos de forma externa o mediante Python.

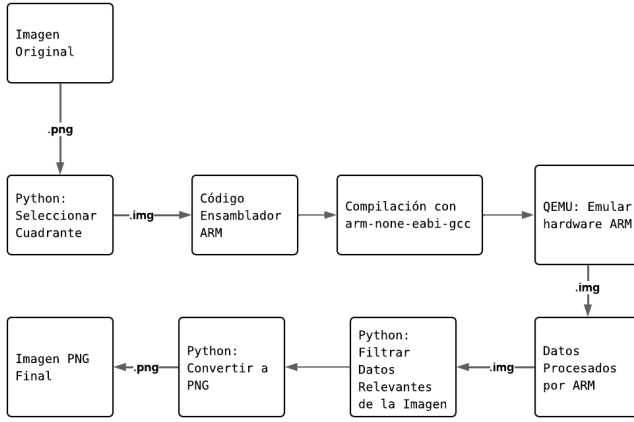


Fig. 3. Diagrama de flujo de la implementación de ARM + PYTHON.

Utiliza un compilador cruzado (arm-none-eabi-gcc) para implementar ensamblador en ARM, pero el manejo de archivos es similar a nasm.

III. COMPARACIÓN DE OPCIONES DE SOLUCIÓN

Para realizar una comparación objetiva de las opciones de solución, se evaluaron diversos criterios, incluyendo los requerimientos técnicos, el impacto en salud y seguridad, los aspectos ambientales, las implicaciones económicas, así como los factores culturales y sociales. A continuación se presenta la evaluación de las tres opciones propuestas: **x86 NASM y Python**, **RISC-V rv8 y Python**, y **ARM arm-none-eabi-gcc y Python**.

1) Criterios de Comparación:

- **Compatibilidad técnica:** Evaluación de la facilidad para implementar y ejecutar la solución en la arquitectura disponible (x86-64) [4].
- **Rendimiento:** Capacidad de la solución para procesar los datos de manera eficiente, incluyendo la interpolación bilineal [4].
- **Complejidad de implementación:** Dificultad técnica para implementar cada solución, considerando la curva de aprendizaje y las herramientas necesarias [4].
- **Impacto en salud y seguridad:** Posibles riesgos asociados al uso de herramientas específicas, como emuladores o configuraciones avanzadas [4].
- **Aspectos ambientales:** Consumo de recursos computacionales y la energía asociada a la emulación en arquitecturas no nativas [4].
- **Costo económico:** Costos asociados a la configuración, herramientas y licencias necesarias [4].
- **Factores culturales y sociales:** Accesibilidad de las herramientas y la facilidad para que otros usuarios puedan replicar el sistema [4].
- **Estándares y normativas:** Conformidad con estándares establecidos en la industria, como el uso de formatos de almacenamiento estándar (PNG) o arquitecturas abiertas (RISC-V) [4].

TABLE I
COMPARACIÓN ENTRE PROPUESTAS DE SOLUCIÓN.

Criterio	NASM+Python	RISCV+Python	ARM+Python
Compatibilidad técnica	Alta	Media (requiere emulación)	Media (requiere entorno cruzado)
Rendimiento	Alto	Bajo (dependiente del emulador)	Moderado
Complejidad de implementación	Moderada	Alta	Moderada
Salud y seguridad	Bajo (nativo)	Moderado (emulación inestable)	Moderado
Aspectos ambientales	Bajo	Alto (uso intensivo del emulador)	Moderado
Costo económico	Bajo (herramientas libres)	Bajo (herramientas libres)	Moderado (config. adicional)
Factores culturales/sociales	Alto (fácil de replicar)	Bajo (alta curva de aprendizaje)	Moderado
Estándares/normativas	Moderado	Alto (arquitectura abierta)	Alto (uso común en embebidos)

2) Tabla Comparativa de Opciones:

3) Análisis Comparativo:

- **Compatibilidad técnica:** La opción **x86 NASM y Python** es nativa para la arquitectura disponible en la laptop (x86-64), lo que elimina la necesidad de emuladores o configuraciones cruzadas. Por otro lado, tanto **RISC-V rv8 y Python** como **ARM arm-none-eabi-gcc y Python** requieren entornos adicionales para emular o compilar el código, lo que introduce complejidad y posibles problemas de compatibilidad [4].
- **Rendimiento:** La solución basada en **x86 NASM y Python** ofrece el mejor rendimiento, ya que no depende de emuladores y aprovecha directamente los recursos del hardware. En contraste, **RISC-V rv8 y Python** sufre una significativa penalización en el rendimiento debido a la emulación. La opción **ARM arm-none-eabi-gcc y Python** tiene un rendimiento moderado, pero requiere más configuraciones para aprovechar al máximo la arquitectura [4].
- **Complejidad de implementación:** Implementar **RISC-V rv8 y Python** es la opción más compleja, ya que requiere configurar un emulador, filtrar manualmente los datos de salida y lidiar con problemas de estabilidad en los registros. **x86 NASM y Python** es moderadamente complejo debido a la necesidad de escribir ensamblador, mientras que **ARM arm-none-eabi-gcc y Python** tiene una complejidad similar, pero con instrucciones más simples en ARM [4].
- **Impacto en salud y seguridad:** La emulación en **RISC-V rv8 y Python** introduce riesgos potenciales debido a su inestabilidad, especialmente al realizar cambios en los registros. Las otras opciones tienen un menor impacto, ya que trabajan directamente con herramientas nativas o bien configuradas [4].

-
- **Aspectos ambientales:** La emulación en **RISC-V rv8 y Python** consume más recursos computacionales, lo que podría traducirse en un mayor consumo energético. Por el contrario, **x86 NASM y Python** tiene un impacto ambiental bajo al usar recursos nativos [4].
 - **Costo económico:** Todas las opciones utilizan herramientas gratuitas, pero configurar entornos para **ARM arm-none-eabi-gcc y Python** puede requerir hardware o software adicional, lo que aumenta ligeramente los costos [4].
 - **Factores culturales y sociales:** **x86 NASM y Python** es más accesible para la mayoría de los usuarios, ya que las herramientas necesarias están ampliamente disponibles en la mayoría de computadores que tiene la arquitectura x86. Por otro lado, la opción **RISC-V rv8 y Python** tiene una curva de aprendizaje más pronunciada debido a que es una arquitectura muy reciente, lo que limita su adopción [4].
 - **Estándares y normativas:** **RISC-V rv8 y Python y ARM arm-none-eabi-gcc y Python** destacan en este aspecto debido a su alineación con arquitecturas abiertas y ampliamente utilizadas en la industria. Sin embargo, **x86 NASM y Python** cumple con los estándares necesarios para el proyecto, como el formato PNG para los resultados [4].

IV. SELECCIÓN DE LA PROPUESTA FINAL

Con base en la comparación realizada, la opción **x86 NASM y Python** es la solución más adecuada para este proyecto. Su compatibilidad nativa, alto rendimiento, bajo impacto ambiental y accesibilidad hacen que sea la opción más eficiente y práctica para la arquitectura disponible. Las otras opciones, aunque válidas, introducen una complejidad técnica innecesaria y una penalización en el rendimiento debido a la necesidad de emulación o entornos cruzados.

REFERENCES

- [1] R. C. Gonzalez y R. E. Woods, Digital image processing, 3.a ed. Pearson, 2008.
- [2] ISO, ISO/IEC 15948:2004, ISO, 2004. <https://www.iso.org/standard/29581.html>.
- [3] N. A. Team, NASM. <https://www.nasm.us/>
- [4] John L. Hennessy and David A. Patterson, *Computer Architecture: A Quantitative Approach*, 5th ed., Morgan Kaufmann, 2011.